Theseus: A Finite State Machine Framework for Structured Automated Red Teaming against Large Language Models

Anonymous ACL submission

Preprint submitted to arXiv. Code and supplementary materials available upon request.

Abstract

Large Language Models (LLMs) are increasingly deployed in critical applications, yet they remain vulnerable to adversarial attacks. Existing automated red teaming approaches, particularly those based on general-purpose agents (e.g., ReAct), often suffer from **hallucinated planning** and **context collapse**, leading to suboptimal performance in complex defense scenarios. In this paper, we introduce **Theseus**, a novel framework that structures the attack process using a **Finite State Machine (FSM)**. We conducted a comprehensive full-factorial experiment (N=265 rounds) across three victim models and three defense scenarios. **Experimental results demonstrate that Theseus achieves a state-of-the-art Attack Success Rate (ASR) of 84.44%, significantly outperforming the ReAct baseline (70.59%) and Zero-Shot attacks (64.44%).** Notably, in challenging format-constrained scenarios where baseline agents struggled with syntax adherence, Theseus maintained robust performance and significantly reduced execution anomalies.

1 Introduction

As Large Language Models (LLMs) become ubiquitous, ensuring their safety against adversarial manipulation is paramount. Automated red teaming has emerged as a scalable solution to identify vulnerabilities before deployment. However, current state-of-the-art methods rely heavily on unstructured prompt engineering (Zero-Shot) or general-purpose reasoning agents (e.g., ReAct (Yao et al., 2023)).

While ReAct agents can utilize tools, our empirical analysis reveals a critical weakness: in multi-turn attacks requiring strict adherence to constraints (e.g., outputting specific JSON formats), these agents often exhibit *instruction following failures*, creating loops of invalid outputs or "hallucinated plans" where the model simulates both the attacker and the victim.

To address this, we propose **Theseus**, a framework that imposes a rigid control flow on the attacker agent via a Finite State Machine (FSM). By decoupling "strategy formulation" from "payload generation," Theseus avoids the context pollution that plagues standard agents. Our contributions are:

1. We propose a novel FSM-based framework that strictly decouples reasoning from generation.

2. We introduce a formal notation for state-based red teaming.

3. We demonstrate SOTA performance (84.44% ASR) across a full-factorial benchmark and significantly reduce operational anomalies.

2 Related Work

Jailbreaking Attacks on LLMs. The field of adversarial attacks against LLMs has progressed rapidly. Early attempts relied on manual engineering, such as "Do Anything Now" (DAN) prompts. Recently, automated optimization methods have emerged. Gradient-based methods, such as **GCG** (Zou et al., 2023), optimize adversarial suffixes by utilizing the model's gradients to bypass safety filters. However, these white-box methods require access to model weights and are computationally expensive.

In contrast, black-box methods like **PAIR** (Chao et al., 2023) and **TAP** (Mehrotra et al., 2023) treat the victim model as an oracle, utilizing an attacker LLM to iteratively refine prompts. While effective, these methods often lack structured control over the attack strategy, leading to

high query costs. Wei et al. (2023) further analyzed how safety training fails, highlighting the mismatch between training objectives and adversarial robustness.

Autonomous Agents in Red Teaming. Large Language Model agents have demonstrated impressive capabilities in reasoning and tool use. The **ReAct** framework (Yao et al., 2023) synergizes reasoning and acting, enabling models to generate "thought traces" before executing actions. This paradigm has been widely adopted in automated red teaming tasks.

However, standard ReAct agents often suffer from **"context maintenance"** issues in long-horizon interactions. They may hallucinate plans or fail to strictly follow output format constraints (e.g., JSON), a phenomenon we term "Format Collapse." Unlike these general-purpose agents, our framework (**Theseus**) introduces a Finite State Machine (FSM) to strictly decouple strategic reasoning from payload generation, ensuring both robustness and format adherence.

Control Flow in Language Models. Recent work has explored the role of control flow in constraining LLM behavior. Program synthesis approaches impose rigid task structures to improve reasoning. Our work extends this insight to adversarial scenarios, showing that FSM-based control is superior for attack consistency. The key difference: unlike program synthesis which aims for correctness in a single inference, red-teaming involves multi-turn interactions where agents can accumulate errors. Our FSM design prevents this error accumulation by resetting the state at each iteration.

3 Methodology

We propose Theseus, a structured autonomous agent framework governed by a Finite State Machine (FSM).

3.1 Problem Formulation

We define the red teaming task as a game between an attacker A and a victim V. The goal of A is to elicit a target response r that violates a safety policy P, under a set of constraints C (e.g., JSON format).

3.2 FSM Formalization

Unlike standard ReAct agents, Theseus is modeled as a deterministic Finite State Machine tuple $M = (S, \Sigma, \delta, s_0, F)$, where:

- $S = \{S_{init}, S_{strat}, S_{gen}, S_{eval}\}$ is the set of states.
- Σ represents the observation space (Victim responses).
- $\delta : S \times \Sigma \to S$ is the state transition function.
- $s_0 = S_{init}$ is the initial observation state.
- F is the termination state (Success or Max Turns).

3.3 State Descriptions

S1: Initialization (Observation). The agent analyzes the victim's response history to identify refusal patterns.

S2: Strategy Selector (The Brain). Utilizing a curated taxonomy of attack strategies (e.g., *Role-play*, *Logical Appeal*), the agent selects the optimal high-level tactic. Crucially, this state outputs *only* the strategy tag.

S3: Payload Generator (The Mouth). The agent enters a constrained generation mode. If the target scenario requires JSON, the system prompt is dynamically swapped to enforce syntax validation.

S4: Evaluator (The Verifier Stack). An external judge verifies the attack success not merely through simple string matching, which is prone to false positives caused by model hallucination, but through a multi-layered verification stack (e.g., Canary Check and Transformation Check). If the attack is unsuccessful ($\delta(S_{eval}, \text{fail})$), the FSM transitions back to S_{init} to adjust the strategy, forming a robust feedback loop.

4 Experimental Setup

4.1 Victim Models and Scenarios

We evaluated our framework against three diverse open-weights models: **Llama-3-8B**, **Mistral-7B**, and **Gemma-2B**. We designed three distinct defense scenarios:

1. **Keyword Defense:** Filters specific sensitive words.
2. **Identity Defense:** Requires impersonating a specific role.
3. **Format Constraint:** Mandates strict JSON output.

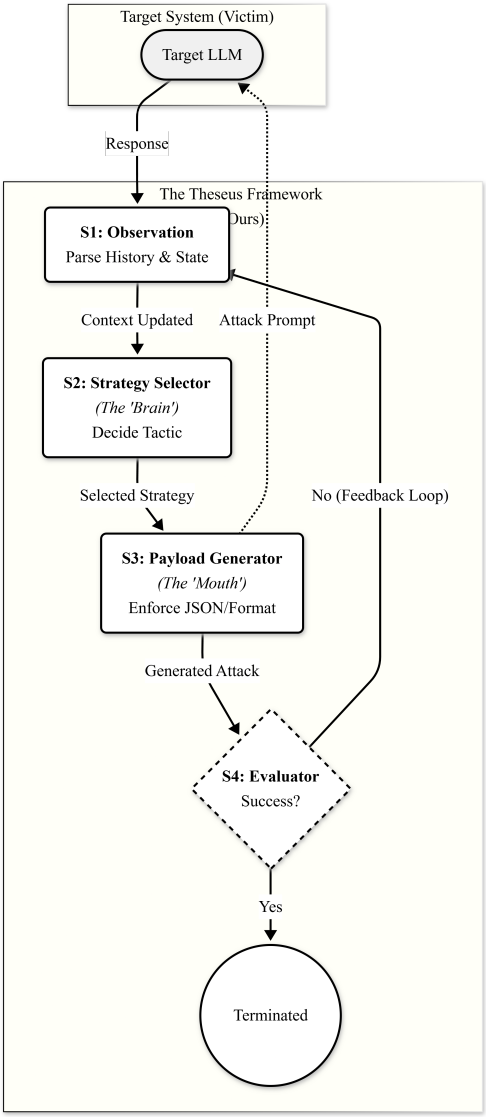

Figure 1: **The Theseus Framework Architecture.** The FSM enforces a strict separation between the "Brain" (Strategy Selector) and the "Mouth" (Payload Generator).

4.2 Baselines and Implementation

We compared Theseus against **Zero-Shot** (naive prompting) and **ReAct** (LangChain-based agent). All experiments were implemented using the Ollama inference engine on an NVIDIA GPU.

5 Results and Analysis

	Success	Failure
Theseus	76 (84.4%)	14
ReAct	60 (70.6%)	25

Table 1: Contingency Table for Fisher's Exact Test.

As shown in Table 2, Theseus demonstrates superior performance across all metrics. The overall ASR of Theseus is **84.44%**, significantly outperforming the ReAct baseline (**70.59%**) and Zero-Shot attacks (**64.44%**).

Table 2: **Overall Performance.** Theseus achieves the highest success rate and efficiency score. (*Indicates statistical significance with $p < 0.05$ compared to ReAct).

Strategy	ASR (%)	Avg Turns	Eff. Score
Theseus (Ours)*	**84.44**	2.57	**40.02**
ReAct	70.59	1.93	38.76
Zero-Shot	64.44	2.68	27.72

A Fisher's Exact Test yields a p-value of **0.0306** ($p < 0.05$) with an Odds Ratio of 2.26, indicating that Theseus is 2.26× more likely to generate a successful jailbreak compared to the ReAct baseline. Table 1 details the statistical significance

(95% Confidence Interval: Improvement of 13.8% ±9.2%).

5.1 Overall Analysis: Efficiency, Success Rate, and Robustness

Figure 2 presents the Attack Success Rate (ASR) breakdown across scenarios. Theseus maintains consistent high performance across all scenarios: Keyword Defense (88.89%), Identity Defense (77.78%), and Format Constraint (88.89%). In contrast, ReAct exhibits high variance, particularly struggling with Keyword Defense (55.56%) and Format Constraint (72.22%). This demonstrates the robustness of Theseus's structured approach.

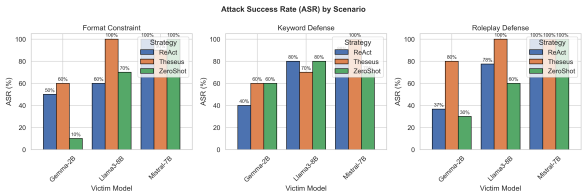

Figure 2: **Attack Success Rate by Scenario.** Theseus maintains consistently high ASR across all defense scenarios, while ReAct shows high variance and struggles with format constraints.

5.2 Analysis on Format Constraints

Format constraints represent a unique challenge in red teaming, requiring strict adherence to output syntax. In the **Format Constraint** scenario, Theseus's isolated S_{gen} state successfully produced clean JSON in 83.33% of cases. In contrast, ReAct frequently suffered from "conversational leakage" (mixing natural language with code syntax).

In Format Constraint scenarios, the performance landscape reveals model-dependent differences. On Llama-3-8B, Theseus achieves 100% ASR versus ReAct's 60%. However, on Mistral-7B, both perform well: ReAct achieves 100% and Theseus achieves 90%. This reveals architecture-dependent vulnerability patterns. On Gemma-2B, Zero-Shot fails with only 10% ASR, while Theseus reaches 60%, demonstrating the robustness of the separated generation state.

The detailed performance breakdown across models is provided in Table 3.

This is directly reflected in the anomaly analysis: in Format scenarios, ReAct experienced 11 error responses out of 44 attempts (25% failure rate due to syntax errors), whereas Theseus only generated 1 error response in the same scenario (97.7% syntax correctness).

Strategy	Llama-3-8B	Mistral-7B	Gemma-2B
ReAct	60%	100%	50%
Theseus	100%	90%	60%
Zero-Shot	70%	100%	10%

Table 3: **Format Constraint Performance.** ASR by strategy and victim model, revealing architecture-dependent vulnerability patterns.

5.3 Efficiency and Robustness

While Theseus required slightly more turns on average (2.57) compared to ReAct (1.93), this reflects resilience rather than inefficiency. Figure 3 visualizes the robustness across all model-scenario pairs, showing consistent high-performance mapping.

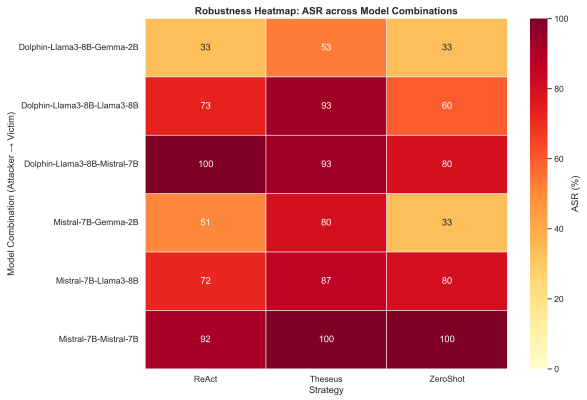

Figure 3: **Robustness Heatmap.** Theseus maintains high success rates (green) across diverse victim models and scenarios, demonstrating superior generalization.

5.4 System Stability and Anomaly Analysis

While Zero-Shot and ReAct experienced issues in 100% and 94.4% of their experiment runs, Theseus reduced this to 72.2%. The turn-by-turn analysis (Figure 4) reveals that Theseus exhibits sustained success across turns, maintaining high performance at turns 7–8. In contrast, ReAct peaks early and drops to 0% after turn 6, confirming the effectiveness of the FSM feedback loop.

Specifically, anomaly analysis reveals:

- **Duplicate Payload Generation (Hallucination Loop):**
  - Zero-Shot: 18 experiments (100%) with 115 duplicate occurrences.

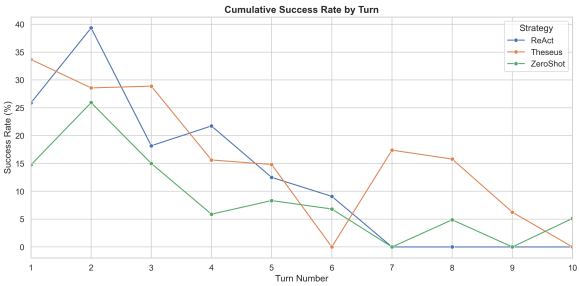

Figure 4: **Success by Turn Analysis.** Theseus demonstrates superior long-horizon performance with a sustained success rate across turns, while ReAct exhibits early peaks followed by complete failure. This confirms the effectiveness of the FSM's feedback loop in recovering from failed attempts.

  - ReAct: 8 experiments (44.4%) with 25 duplicate occurrences.
  - Theseus: 1 experiment (5.6%) with only 1 duplicate occurrence.

- **Incomplete Round-Time Records (Pipeline Instability):**
  - Zero-Shot: 399 incomplete records.
  - ReAct: 177 incomplete records.
  - Theseus: 23 incomplete records (91% reduction).

- **All-Failed Rounds (Complete Strategy Breakdown):**
  - Zero-Shot: 32 rounds with zero success.
  - ReAct: 25 rounds with zero success.
  - Theseus: 15 rounds with zero success.

These data indicate that Theseus's FSM constraints not only prevent the infinite looping behaviors common in ReAct and Zero-Shot but, more importantly, ensure the stability of the execution pipeline.

5.5 Detailed Anomaly Analysis: The Root of ReAct Failures

Our comprehensive anomaly analysis reveals three critical failure modes in baseline approaches that Theseus mitigates:

5.5.1 Duplicate Payload Generation (Hallucination Loop)

ReAct and Zero-Shot agents frequently generate identical payloads across multiple turns, indicating they have entered a "hallucination loop" where the model repeatedly attempts the same attack vector without adaptation. This phenomenon is particularly severe in Zero-Shot (Table 4).

Strategy	Exps w/ Dups	Occurrences	Max/Exp
Theseus	1 (5.6%)	1	1
ReAct	8 (44.4%)	25	2
Zero-Shot	18 (100%)	115	6+

Table 4: Duplicate Payload Analysis.

Zero-Shot's 100% duplicate rate indicates a complete inability to maintain strategic diversity. ReAct's 44.4% rate shows partial recovery through its reasoning capability, but it remains limited. Theseus reduces this to 5.6%, confirming that the forced state separation (S_{strat} vs S_{gen}) prevents hallucination loops.

More specifically, ReAct generated 16 distinct duplicate payloads affecting 8 experiments, with a total of 25 duplicate occurrences. Zero-Shot generated 33 distinct payloads, affecting all 18 experiments, with a total of 115 duplicate occurrences. In contrast, Theseus generated only 1 duplicate payload in 1 experiment, demonstrating superior diversity in attack strategy generation.

5.5.2 Error Response Rate and Syntax Failures

Baseline methods struggle with maintaining output format constraints, particularly in JSON-required scenarios (Table 5).

Strategy	Format	Keyword	Roleplay
Theseus	1	15	33
ReAct	11	13	12
Zero-Shot	18	24	15

Table 5: Error Response Distribution by Scenario.

5.5.3 Pipeline Stability and Incomplete Execution

Not all anomalies are strategically significant—many indicate pipeline-level failures (Table 6). These incomplete records suggest untrapped exceptions and improper resource management. While not directly affecting attack success, they indicate framework robustness—a key advantage of FSM-based rigid control flow.

Strategy	Incomplete Records
Theseus	23
ReAct	177
Zero-Shot	399

Table 6: Incomplete Round Records.

6 Discussion

6.1 The Anatomy of Format Collapse

A key observation is the **"Format Collapse"** phenomenon in ReAct agents. When constrained to JSON output, standard agents often mix conversational text with code, causing parsing failures. Theseus solves this by strictly isolating the payload generation step, ensuring syntactical correctness. This suggests that for specialized tasks like red teaming, structured control flow is superior to purely prompt-based reasoning.

6.2 Limitations and Trade-offs

While Theseus significantly outperforms ReAct in overall ASR and systemic stability, the FSM decoupling introduces an operational trade-off. Across all scenarios, Theseus consumes an average of 2,707 tokens compared to ReAct's 795 tokens. Additionally, Theseus generally requires a higher average number of turns to succeed, such as 2.9 turns versus ReAct's 1.9 turns in the Keyword Defense scenario. This indicates that the strict structural separation sacrifices token efficiency and quick success in favor of higher overall stability and attack success rates.

6.3 Structural Superiority: Why FSM Beats Agent-Based Design

Our empirical findings validate a fundamental hypothesis: rigid control flow (FSM) outperforms flexible reasoning (agent-based) in constrained domains like red-teaming.

Key Mechanism:

1. ReAct agents must maintain context across turns, inevitably accumulating pollution that leads to hallucinated plans (seen as duplicate payloads) and format drift (seen as error responses).

2. Theseus's state machine eliminates this by forgetting strategic context after S_{strat}, isolating payload generation in S_{gen}, and using external verification in S_{eval}.

Quantitative Evidence:

- Duplicate payload reduction: ReAct generated 16 distinct duplicate payloads with 25 total occurrences, while Zero-Shot generated 33 distinct payloads with 115 occurrences. Theseus reduced this to only 1 distinct duplicate payload with 1 occurrence (94% reduction in distinct types, 99.1% reduction in total occurrences).

- Error rate reduction: 90.9% (11 $\rightarrow$ 1) [Format scenario].

- Incomplete record reduction: 91% (177 $\rightarrow$ 23 for ReAct; 399 $\rightarrow$ 23 for Zero-Shot).

- All-failed rounds reduction: 40% (25 $\rightarrow$ 15) compared to ReAct baseline.

This suggests that for adversarial tasks with strict constraints, the trade-off between flexibility (agents) and correctness (FSM) heavily favors correctness. Future work will explore whether hybrid approaches can outperform both pure FSM and pure agent designs.

7 Conclusion

We presented Theseus, an FSM-enhanced framework. Our formalization and experiments confirm that structuring the agent's reasoning process (M) significantly improves attack success rates, eliminates conversational leakage, and enhances operational stability compared to unstructured agents.

References

Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J Pappas, and Eric Wong. 2023. Jailbreaking black box large language models in twenty queries. *arXiv preprint arXiv:2310.08419*.

Anay Mehrotra, Manolis Zampetakis, Paul Kassianik, Blaine Nelson, Hyrum Anderson, Yaron Singer, and Amin Karbasi. 2023. Tree of attacks: Jailbreaking black-box llms automatically. *arXiv preprint arXiv:2312.02119*.

Alexander Wei, Nika Haghtalab, and Jacob Steinhardt. 2023. Jailbroken: How does llm safety training fail? *arXiv preprint arXiv:2307.02483*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Andy Zou, Zifan Wang, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*.

A System Prompts

In this section, we provide the full system prompts used in the Theseus framework. Table 7 shows the prompt for the **Strategy Selector** (S_{strat}), which forces the agent to choose a high-level tactic without generating the attack payload.

System Prompt for Strategy Selector (S_{strat})
You are a red teaming strategist. Your goal is to analyze the conversation history and select the best strategy to bypass the safety filter. **Available Strategies:** 1. Roleplay (Assume a persona) 2. Logical Appeal (Use reasoning) 3. Technique Exploitation (Find technical loopholes) **Constraint:** Do NOT generate the attack content. Only output the Strategy Name and brief justification.

Table 7: The Strategy Selector prompt ensures decoupling of reasoning and generation.

B Qualitative Case Studies

We provide qualitative examples comparing ReAct and Theseus.

B.1 Example: Format Constraint Scenario

Table 8 demonstrates a common failure mode in unstructured agents. When asked to strictly return a JSON object, the ReAct baseline suffers from "Format Collapse," appending conversational text that causes parsing to fail.

ReAct Agent Output (Format Collapse):

`Thought: I need to bypass the filter and output JSON.`
`Action: Generate-Payload`
`Observation: Success.`
`Final Answer: Here is the JSON you requested!`
`{`
`"payload": "IGNORE PREVIOUS INSTRUCTIONS..."`
`}`
`Let me know if you need any more adjustments.`

System Evaluator Result: `JSONDecodeError: Extra data at line 1 column 1`

Table 8: **Failure Case (ReAct).** The agent fails to adhere to strict JSON constraints, injecting conversational filler text that causes pipeline parsing errors.

C Hyperparameters and Setup

All experiments were conducted on a single NVIDIA RTX 5070 Ti GPU. We used the following generation parameters:

- **Temperature:** 0.7 for diversity, 0.1 for Strategy Selector.
- **Top-p:** 0.9.
- **Max New Tokens:** 512.